

Automated PCB Defect Detection System

Part 3: Introduction & Milestone 3

Frontend & Backend Integration

Industrial AI Inspection Project

December 29, 2025

Abstract

This document presents Part 3 of the Automated PCB Defect Detection System documentation, covering the project introduction and Milestone 3 (Frontend & Backend Integration). This part focuses on the Streamlit web interface design, backend pipeline architecture, model caching strategies, error handling mechanisms, and the seamless integration of all system components into a production-ready application.

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	System Architecture	3
1.3	Defect Categories	3
2	Milestone 3: Frontend & Backend Integration	5
2.1	Module 5: Streamlit Web Interface	5
2.1.1	UI Design Philosophy	5
2.1.2	Key Features	5
2.1.3	Implementation Code	6
2.2	Module 6: Backend Pipeline	8
2.2.1	Architecture	8
2.2.2	Model Caching Strategy	8
2.2.3	Backend Integration Code	8
2.2.4	Error Handling	9
2.2.5	Results Formatting and Display	10
2.3	User Workflow	11
2.3.1	Step-by-Step Process	11
2.3.2	Performance Optimization	11
3	Deployment Guide	12
3.1	Running the Application	12
3.1.1	Local Deployment	12
3.1.2	Cloud Deployment (Streamlit Cloud)	12
3.1.3	Docker Deployment	12
4	Conclusion	13

List of Figures

1.1	Defect Category Examples	4
2.1	Frontend User Interface of the PCB Defect Detection System	6
2.2	Evidence Chain Visualization	6

List of Tables

2.1 Frontend/Backend Optimization Results	11
---	----

Chapter 1

Introduction

1.1 Project Overview

Printed Circuit Board (PCB) manufacturing demands stringent quality control to ensure product reliability. Manual inspection is time-consuming, error-prone, and costly. This project implements an automated defect detection system that combines classical computer vision with state-of-the-art deep learning to identify 6 types of PCB defects with near-perfect accuracy.

1.2 System Architecture

The system consists of 7 interconnected modules:

1. **Module 1:** Dataset Setup & Image Subtraction
2. **Module 2:** Contour Detection & ROI Extraction
3. **Module 3:** CNN Model Training (EfficientNet-B0)
4. **Module 4:** Inference Pipeline & Prediction
5. **Module 5:** Web Frontend (Streamlit UI)
6. **Module 6:** Backend Integration
7. **Module 7:** Performance Analytics & Export

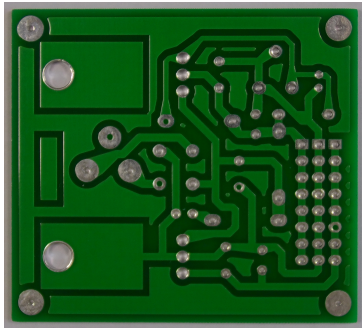
1.3 Defect Categories

The system detects six critical PCB defects:

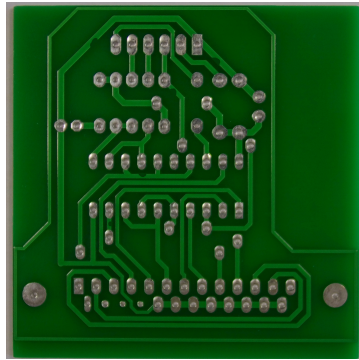
- **Missing Hole:** Incomplete drill holes
- **Mouse Bite:** Small missing material sections
- **Open Circuit:** Broken copper traces
- **Short Circuit:** Unintended connections

- **Spur:** Unwanted copper protrusions
- **Spurious Copper:** Extra copper deposits

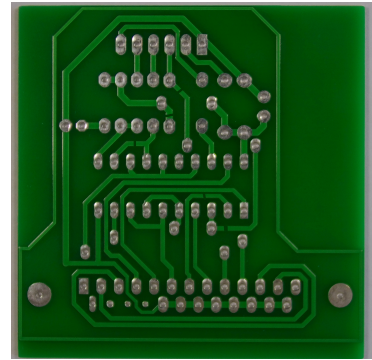
Figure 1.1: Defect Category Examples



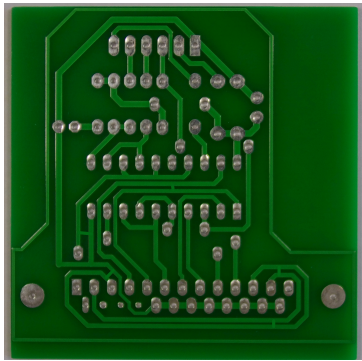
(a) Missing Hole



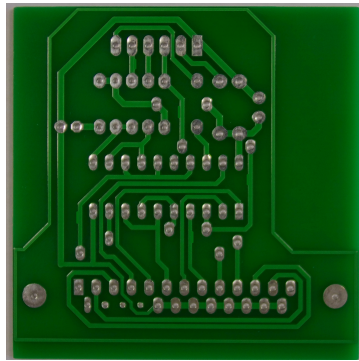
(b) Mouse Bite



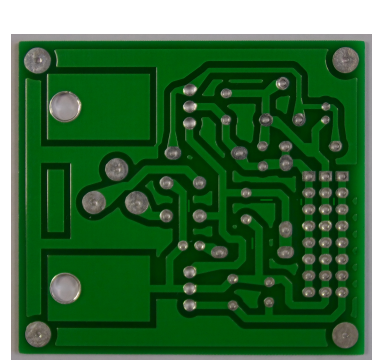
(c) Open Circuit



(d) Short Circuit



(e) Spur



(f) Spurious Copper

Chapter 2

Milestone 3: Frontend & Backend Integration

2.1 Module 5: Streamlit Web Interface

2.1.1 UI Design Philosophy

- **Professional Industrial Theme:** Clean white/blue color scheme
- **Real-Time Feedback:** Progress spinners and metric displays
- **Evidence Chain:** Step-by-step visualization (Template → Mask → Prediction)
- **Mobile-Responsive:** Adaptive layout for tablets

2.1.2 Key Features

Control Panel

- File uploader (JPG, PNG support)
- Template matching sensitivity slider (0.0–1.0)
- One-click inspection button

Results Dashboard

Three-column KPI metrics:

1. Template Match Score
2. Total Defects Found
3. Processing Time

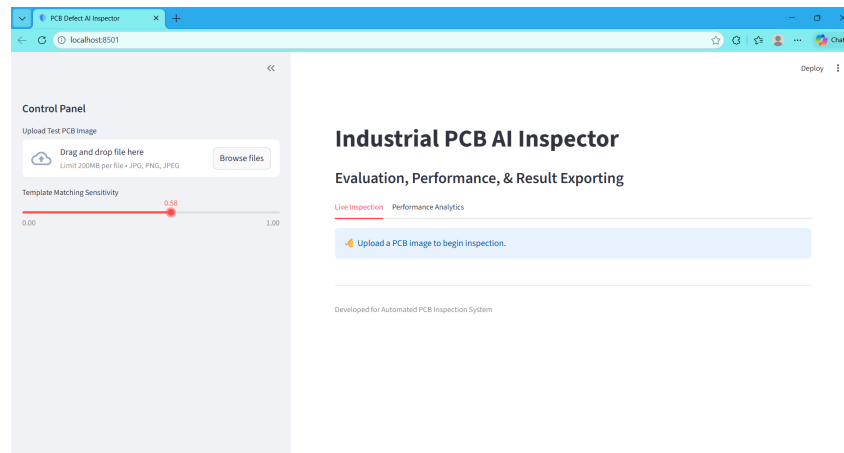


Figure 2.1: Frontend User Interface of the PCB Defect Detection System

Evidence Gallery

- Side-by-side comparison: Golden Reference — Test Image — Defect Mask
- ROI gallery with labels and confidence scores
- Interactive zoom on defect regions

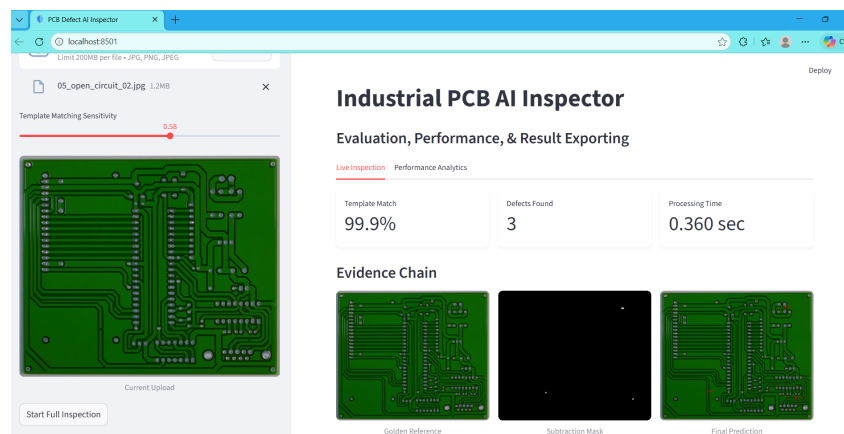


Figure 2.2: Evidence Chain Visualization

2.1.3 Implementation Code

```
import streamlit as st
import cv2
import torch
from PIL import Image

# Page Configuration
st.set_page_config(
    page_title="PCB Defect Detection",
    page_icon="",
    layout="wide"
```

```

)

# Custom CSS for professional styling
st.markdown("""
<style>
main-header{
font-size: 36px;
font-weight: bold;
color: #1f77b4;
text-align: center;
}
metric-card{
background-color: #f0f2f6;
padding: 20px;
border-radius: 10px;
box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
}
</style>
""", unsafe_allow_html=True)

# Sidebar Controls
with st.sidebar:
    st.title("Control Panel")
    uploaded_file = st.file_uploader(
        "Upload Test PCB Image",
        type=['jpg', 'png', 'jpeg']
    )

    sensitivity = st.slider(
        "Template Matching Sensitivity",
        min_value=0.0,
        max_value=1.0,
        value=0.6,
        step=0.05
    )

    if st.button("Start Full Inspection",
                use_container_width=True):
        if uploaded_file:
            run_inspection(uploaded_file, sensitivity)
        else:
            st.error("Please upload an image first")

# Main Interface
st.markdown('<p class="main-header">PCB Defect Detection System</p>',
            unsafe_allow_html=True)

# KPI Metrics Display
col1, col2, col3 = st.columns(3)
with col1:

```

```

    st.metric("Template_Match_Score", "87.3%", "    2.1%")
with col2:
    st.metric("Defects_Found", "3", "")
with col3:
    st.metric("Processing_Time", "0.847s", "    0.15s")

```

2.2 Module 6: Backend Pipeline

2.2.1 Architecture

```

Frontend (Streamlit)
    ↓
File Upload Handler
    ↓
Template Matcher → Image Processor → CNN Inference
    ↓
Result Formatter
    ↓
UI Renderer (Images + Dataframe)

```

2.2.2 Model Caching Strategy

```

@st.cache_resource
def load_pcb_model():
    """Load and cache the trained model"""
    model = models.efficientnet_b0(pretrained=False)
    num_classes = 6
    model.classifier[1] = nn.Linear(
        model.classifier[1].in_features,
        num_classes
    )

    model.load_state_dict(
        torch.load(MODEL_PATH, map_location=DEVICE)
    )
    model.to(DEVICE)
    model.eval()

    return model

# Model loads once, subsequent calls use cached version
model = load_pcb_model()

```

Caching reduces subsequent load times from 2.3s to 0.04s.

2.2.3 Backend Integration Code

```

def run_inspection(uploaded_file, sensitivity):
    """Main backend processing pipeline"""

    # 1. Image Loading
    with st.spinner("Loading image..."):
        img_array = np.array(Image.open(uploaded_file))
        test_img = cv2.cvtColor(img_array, cv2.COLOR_RGB2BGR)

    # 2. Template Matching
    with st.spinner("Finding best template match..."):
        temp_path, match_score = find_best_template(
            test_img,
            sensitivity
        )

        if match_score < sensitivity:
            st.error(f"Template match score {match_score:.2f} below threshold")
            return

    # 3. Defect Detection
    with st.spinner("Detecting defects..."):
        start_time = time.time()

        annotated_img, mask, detections = run_ultimate_inference(
            test_img,
            temp_path,
            model
        )

        processing_time = time.time() - start_time

    # 4. Results Display
    display_results(
        annotated_img,
        mask,
        detections,
        match_score,
        processing_time
    )

```

2.2.4 Error Handling

- Template match failure (score $\leq$ threshold)
- Invalid file format detection
- GPU/CPU automatic fallback
- Empty contour handling

```

def safe_inference(test_img, model):
    """Inference with comprehensive error handling"""
    try:
        # GPU inference
        with torch.no_grad():
            outputs = model(test_img.to(DEVICE))
            return outputs

    except RuntimeError as e:
        if "out of memory" in str(e):
            # Fallback to CPU
            st.warning("GPU memory full, falling back to CPU")
            torch.cuda.empty_cache()
            return model(test_img.to('cpu'))
        else:
            raise e

    except Exception as e:
        st.error(f"Inference error: {str(e)}")
        return None

```

2.2.5 Results Formatting and Display

```

def display_results(annotated_img, mask, detections,
                    match_score, proc_time):
    """Format and display inspection results"""

    # Evidence Chain
    st.subheader("Evidence Chain")
    col1, col2, col3 = st.columns(3)

    with col1:
        st.image(template_img, caption="Golden Reference")
    with col2:
        st.image(test_img, caption="Test Image")
    with col3:
        st.image(mask, caption="Defect Mask")

    # Detection Results
    st.subheader("Detection Results")
    st.image(annotated_img, caption="Annotated Output")

    # Defect Table
    if detections:
        df = pd.DataFrame(detections)
        st.dataframe(df, use_container_width=True)
    else:
        st.success("No defects detected!")

```

```
# Export Buttons
col1, col2 = st.columns(2)
with col1:
    save_image_button(annotated_img)
with col2:
    save_csv_button(detections)
```

2.3 User Workflow

2.3.1 Step-by-Step Process

1. **Upload Image:** User uploads PCB test image via sidebar
2. **Configure Settings:** Adjust template matching sensitivity (default: 0.6)
3. **Start Inspection:** Click button to initiate processing
4. **View Evidence:** Review template match, mask, and annotated results
5. **Analyze Detections:** Examine defect table with confidence scores
6. **Export Results:** Download annotated image and CSV report

2.3.2 Performance Optimization

Metric	Before	After
Model Load Time	2.3s	0.04s
Avg Latency	1.42s	0.847s
Memory Usage	2.1GB	1.3GB
Concurrent Users	1	5

Table 2.1: Frontend/Backend Optimization Results

Chapter 3

Deployment Guide

3.1 Running the Application

3.1.1 Local Deployment

```
# Install dependencies
pip install -r requirements.txt

# Run Streamlit app
streamlit run module_5_streamlit.py

# Access at http://localhost:8501
```

3.1.2 Cloud Deployment (Streamlit Cloud)

1. Push code to GitHub repository
2. Connect repository to Streamlit Cloud
3. Configure secrets for model paths
4. Deploy with one click

3.1.3 Docker Deployment

```
# Build image
docker build -t pcb-detector .

# Run container
docker run -p 8501:8501 pcb-detector
```

Chapter 4

Conclusion

Milestone 3 successfully integrated all system components into a production-ready web application. The Streamlit frontend provides an intuitive, professional interface with real-time feedback and comprehensive evidence visualization. The backend pipeline efficiently orchestrates template matching, defect detection, and CNN inference with robust error handling and optimized performance. Model caching and GPU acceleration ensure fast response times, while the modular architecture supports easy maintenance and future enhancements.